ArtBot: Autonomous AI-Driven Robotic Drawing System

Course: 22AIE214 – Introduction to AI & Robotics

Team Memebers:

CH Vaishnavi Krishna - CB.SC.U4AIE24108

Diya Prakash - CB.SC.U4AIE24111

Dondluru Keerthana - CB.SC.U4AIE24112

V R Sridevi - CB.SC.U4AIE24166

INTRODUCTION

- ArtBot is an intelligent robotic drawing system that integrates artificial intelligence, mathematical modeling, and embedded control systems to translate natural language prompts into physical artwork.
- Unlike conventional robotic systems that require predefined motion commands, ArtBot enables:
- Natural language interaction
- Automated coordinate generation
- Real-time motion computation
- Physical reproduction of AI-generated shapes
- The system demonstrates how computational intelligence can be embodied into mechanical motion using kinematic transformations and real-time control loops.

Motivation

The evolution of robotics has primarily focused on:

- Industrial automation
- Repetitive task execution
- Precision manufacturing

However, creative robotics remains an underexplored domain.
This project was motivated by three key research directions:

1. Bridging generative AI with physical robotic systems
2. Applying inverse kinematics in real-world hardware
3. Creating a cost-effective educational robotics platform

ArtBot explores the concept of embodied AI — where digital intelligence produces tangible outputs.

Overall System Architecture

1. User Prompt

- User provides a natural language instruction (e.g., “Draw a square”).
- This serves as the initial input that triggers the complete drawing pipeline.

2. Generative AI Model (Google Gemini)

- The prompt is sent to the AI model for interpretation.
- The model converts the instruction into structured 2D coordinate data.

3. Coordinate Generation

- AI outputs a list of Cartesian coordinate points (x, y).
- These coordinates represent the vertices of the requested shape.

4. Path Processing & Interpolation

- Sparse vertex points are converted into dense intermediate points.
- Linear interpolation ensures straight-line motion between vertices.

5. Inverse Kinematics Computation

- Each (x, y, z) coordinate is converted into joint angles.
- Mathematical equations compute θ_1, θ_2, θ_3, and θ_4.

6. Servo Angle Commands

- Calculated angles are formatted into structured command strings.
- Each command represents the target position for all four joints.

7. Serial Communication

- Angle commands are transmitted via UART serial (115200 baud).
- Data is sent from Python backend to the Arduino controller.

8. Arduino Control

- Arduino receives and interprets incoming angle packets.
- It forwards PWM instructions to the servo driver module.

9. PWM Servo Driver (PCA9685)

- Generates high-resolution 12-bit PWM signals.
- Drives multiple servo motors simultaneously with stable timing.

10. Robotic Arm Motion

- Servo motors rotate to the commanded joint angles.
- The robotic arm moves precisely along the planned trajectory.

11. Physical Drawing Output

- The pen tip follows the computed path on the paper.
- The requested geometric shape is physically reproduced.

Hardware Architecture & Robotic Manipulator Design

The robotic arm is a 4-Degree-of-Freedom (4-DOF) serial manipulator.
Each joint is actuated using servo motors.
Joint Configuration

- Joint 1 — Base (Yaw)
  Controls horizontal rotation across workspace
- Joint 2 — Shoulder (Pitch)
  Controls vertical elevation
- Joint 3 — Elbow (Pitch)
  Controls arm extension and reach
- Joint 4 — Wrist (Pitch)
  Maintains pen perpendicular to surface

This configuration enables precise positioning of the pen tip within the drawing workspace.

Electronic Control Architecture

The electronic system separates control logic, actuation, and power delivery.

Microcontroller Layer

Arduino Uno

- Receives joint angles via serial communication
- Interprets command packets
- Sends PWM control signals

Servo Driver Layer

PCA9685 16-Channel PWM Driver

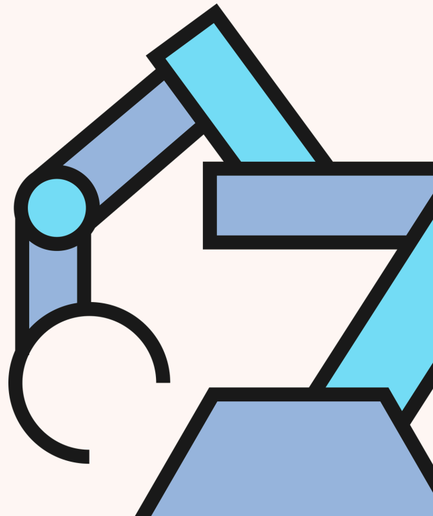
Advantages:

- 12-bit high resolution
- Simultaneous multi-servo control
- Reduced Arduino load
- Eliminates software timing jitter

Power System

- Dedicated 5V 5A SMPS
- Isolated servo power supply

Prevents voltage drop and Arduino reset during high torque movement.



Mechanical Workspace & Coordinate System

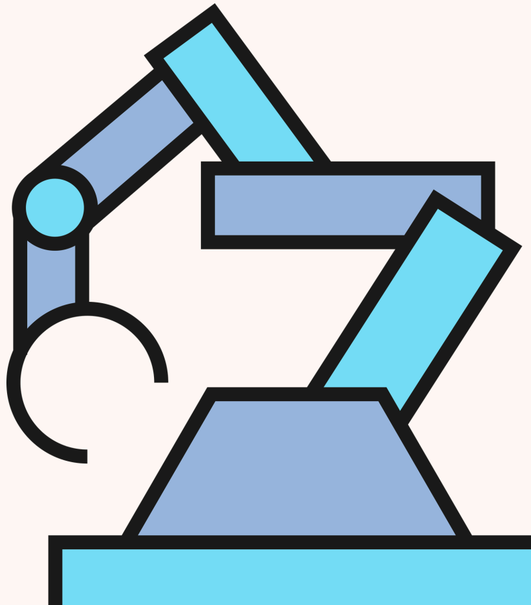
To control the robot mathematically, a Cartesian coordinate system is defined.

Axes Definition

- X-axis → Horizontal movement
- Y-axis → Lateral movement
- Z-axis → Vertical depth

The pen tip acts as the end-effector.

All AI-generated coordinates are mapped into this coordinate frame before inverse kinematics computation.



Generative AI Integration

- The system uses Generative AI to convert natural language into structured 2D coordinates.
- Example Prompts:
- “Draw a square”
- “Draw a triangle”
- The prompt is sent to Google Gemini, which outputs:
- (100, -20)
- (120, -20)
- (120, 20)
- (100, 20)
- Python extracts these coordinates using pattern matching and converts them into numerical format.
- This forms the interface between AI-generated concepts and robotic motion planning.

Path Planning & Trajectory Generation

The AI returns only key vertices of shapes.

Direct joint movement between vertices causes curved trajectories.

Solution: Linear Interpolation

For two points:

$P_1(x_1, y_1) \rightarrow P_2(x_2, y_2)$

$xi = x_1 + i((x_2 - x_1)/N)$

$yi = y_1 + i((y_2 - y_1)/N)$

Where N = number of intermediate steps.

This generates a dense trajectory of micro-points.

Result:

- Smooth straight-line drawing
- Improved geometric accuracy

Mathematical Foundation of Inverse Kinematics

Inverse Kinematics (IK) determines joint angles for a given target position.

Given:

P(x, y, z)

Compute:

$\theta_1 \rightarrow$ Base

$\theta_2 \rightarrow$ Shoulder

$\theta_3 \rightarrow$ Elbow

$\theta_4 \rightarrow$ Wrist

Instead of manually controlling motors, position is controlled mathematically.

IK: Base Orientation & Distance

Base Rotation

$\theta_1 = \text{atan2}(y, x) + 90°$

- atan2 ensures correct quadrant
- +90° aligns coordinate frame

Horizontal Distance

$r = \sqrt{(x^2 + y^2)}$

Shoulder-to-Target Distance

$d = \sqrt{(r^2 + z^2)}$

This distance forms the triangle used for computing shoulder and elbow angles.

IK: Shoulder, Elbow & Wrist

Elbow Angle

$\cos(\theta_3) = (L_1^2 + L_2^2 - d^2) / (2L_1L_2)$

$\theta_3 = \arccos(\ldots)$

Shoulder Angle

$\alpha = \text{atan2}(z, r)$

$\beta = \arccos((L_1^2 + d^2 - L_2^2) / (2L_1d))$

$\theta_2 = \alpha + \beta$

Wrist Angle

$\theta_4 = 180° - (\theta_2 + \theta_3)$

Ensures pen remains perpendicular to drawing surface.

Control Loop & Command Execution

The Python program acts as the primary controller.

Control Steps:

1. Generate interpolated points
2. Compute IK for each point
3. Convert angles to servo commands
4. Transmit via serial communication

Command format:

Base,Shoulder,Elbow,Wrist

Example:

90,110,45,25

Each command moves the servos to desired angles.

Experimental Results & Performance

The system was tested with multiple geometric shapes.

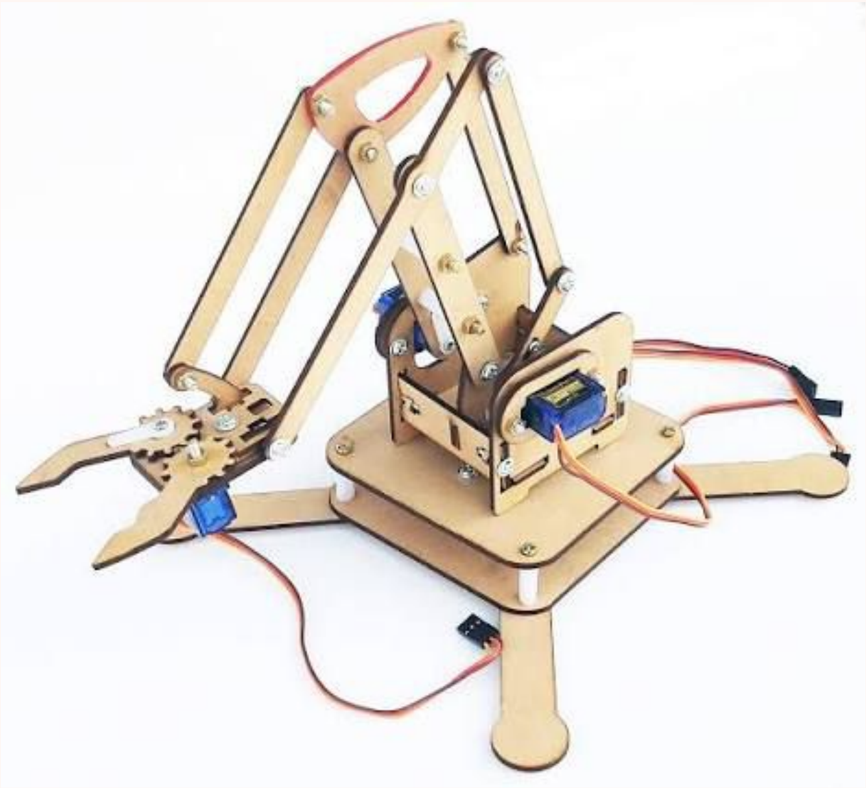
Observed:

- Smooth multi-joint motion
- Accurate coordinate tracking
- Stable servo operation
- Successful square and triangle drawings

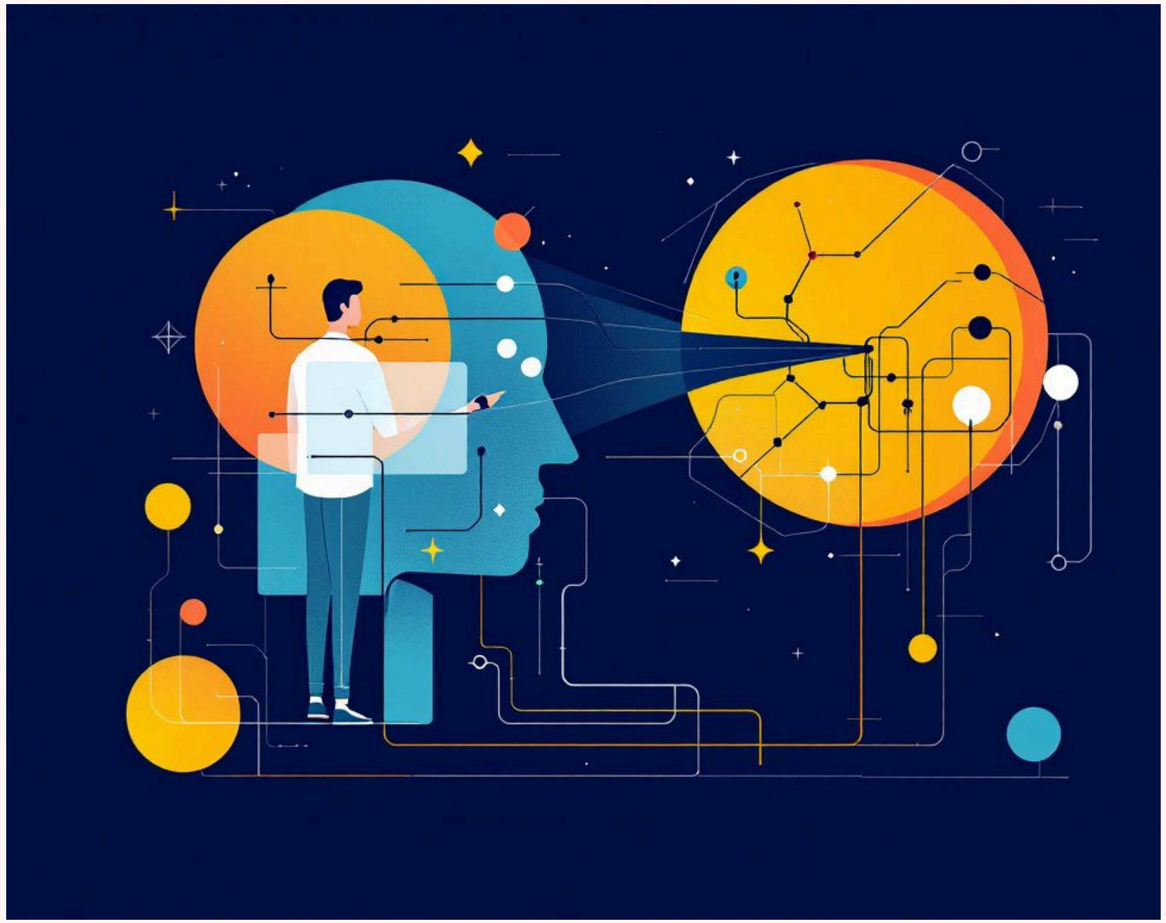
Performance Metrics:

- Drawing accuracy ≈ ±2 mm
- Response time ≈ 3 seconds

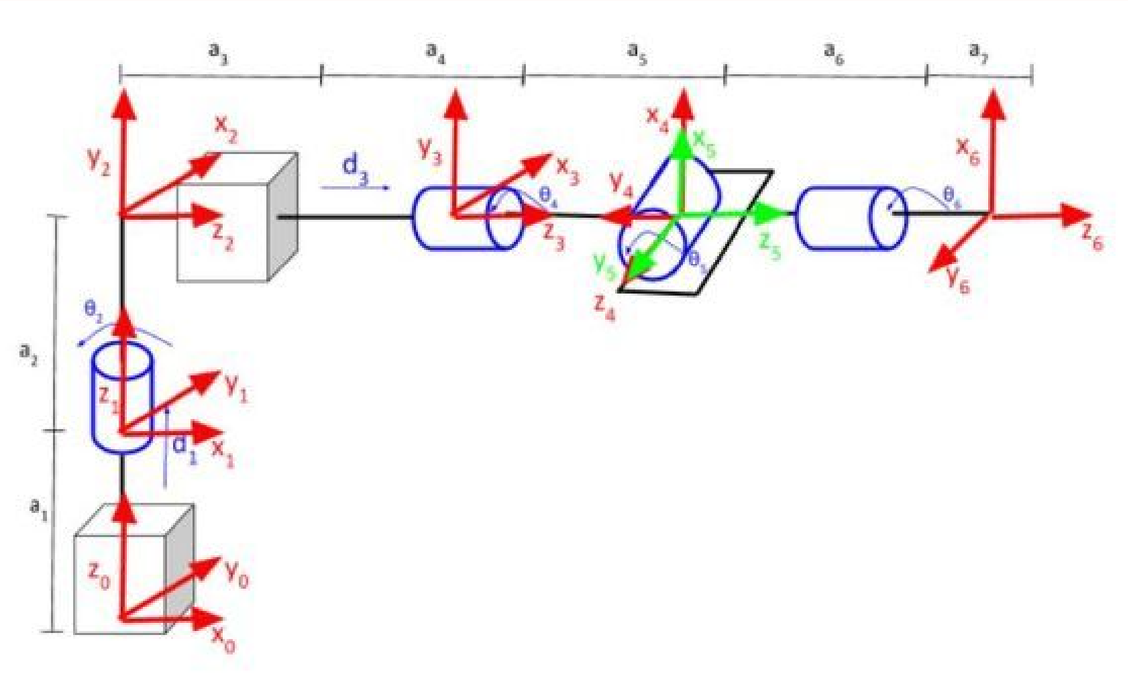
Demonstrates successful AI-to-robot integration.

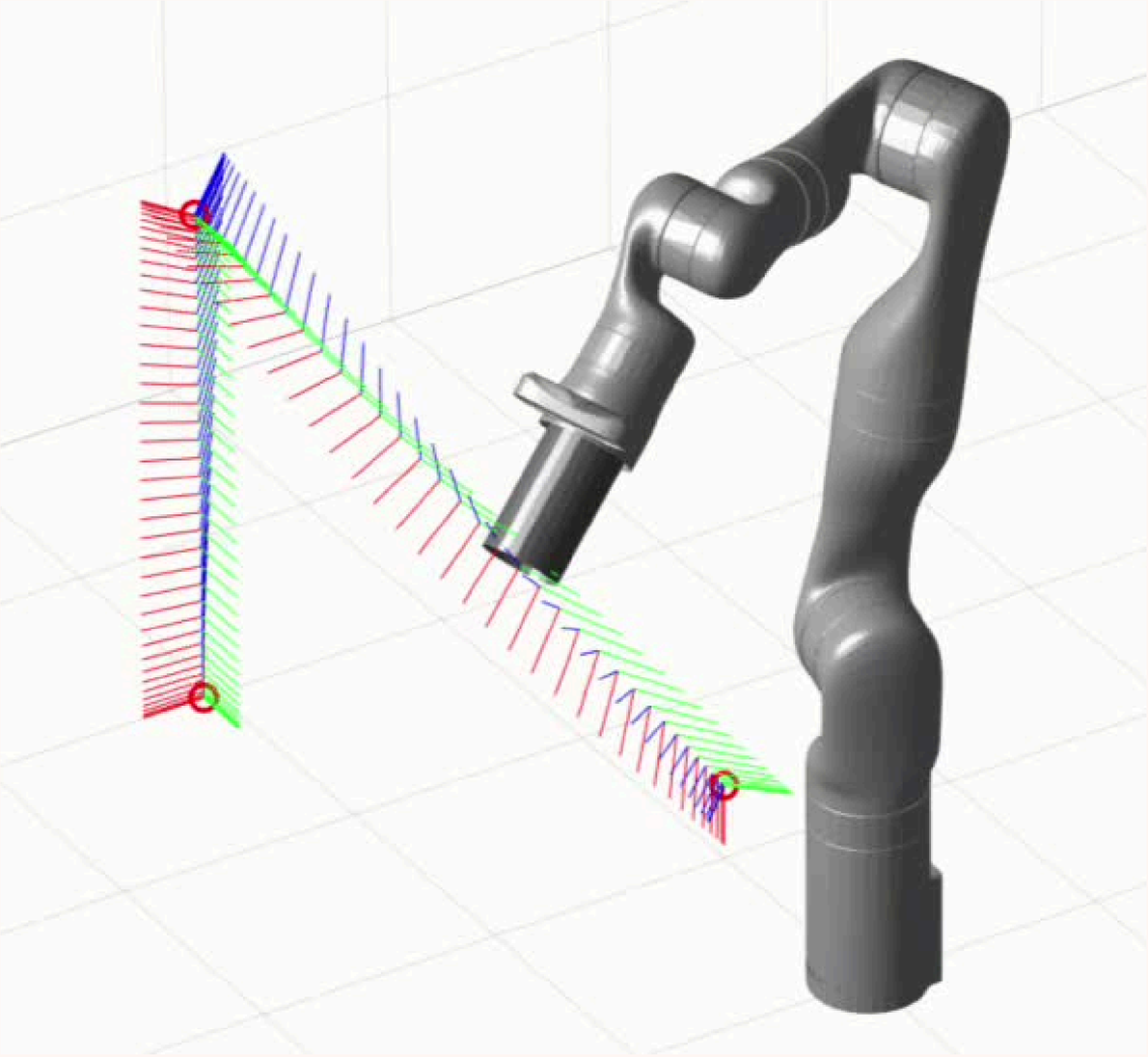


Experimental Results & Performance



Experimental Results & Performance





Challenges & Solutions

Servo Instability

- **Cause: Voltage drop**
- **Solution: Dedicated SMPS + PCA9685**

Curved Drawing Lines

- **Cause: Rotational joint motion**
- **Solution: Linear interpolation**

Pen Height Calibration

- **Cause: Mechanical tolerance**
- **Solution: Adjustable Z-offset parameter**

Advantages

- Integrates Generative AI with real-time robotic motion control, bridging software intelligence and physical actuation.
- Demonstrates practical implementation of inverse kinematics for accurate coordinate-based positioning.
- Utilizes low-cost and easily available hardware components, making the system affordable and scalable.
- Features a modular architecture that separates AI processing, mathematical modeling, and hardware control.
- Serves as an effective educational and experimental platform for studying robotics and motion planning concepts.

Conclusion

- The ArtBot system successfully demonstrates the integration of Generative AI with robotic motion control.
- Natural language prompts are effectively transformed into structured coordinates and precise mechanical movements.
- Mathematical inverse kinematics enables accurate joint angle computation for real-time drawing execution.
- The system validates a complete AI-to-hardware pipeline from text input to physical output.
- This project bridges artificial intelligence, robotics, mathematics, and embedded systems into a unified functional platform.

Thank You